Advanced Data Management: Project Report

Student Name: Ameet Kumar (922870)

Course: Advanced Data Management

Instructor: Prof. Andrea Campagner

Date: 20 June 2025

Database Technologies Used:

- Relational DBMS (MySQL)
- Document DBMS (MongoDB)
- Graph DBMS (Neo4j)
- Distributed Processing (Apache Spark) using:
- DataFrames + SparkSQL
- GraphFrames
- RDDs

At the end of report, there is comparison performance between different database technologies for various type of queries on various possible combination of data.

But for now, to answer exercises we have used following data combination:

To answer all exercise in part 1, we have randomly generated 1000 person record, 10 companies, 25 relationship between person and company.

To answer all exercise in part 2, we have randomly generated 100 users and 990 follow relationships between them which is 10% of all possible combination of 100 users.

1. MySQL Data Insertion – Employment Dataset

Files Used:

- `persons_data/persons_1000.xlsx`
- `companies_data/companies_10.xlsx`
- `relation_data/person_company_relationship.xlsx`

Core Logic:

You use **SQLAlchemy** to connect to a MySQL database and insert Excel data into the `person`, `company`, and `works_in` tables.

Key Steps:

```
def load_excel_file(file_path):
    return pd.read_excel(file_path)
```

```
def insert_data(engine, table_name, df):
    for _, row in df.iterrows():
        stmt = text(f"INSERT INTO {table_name} (...) VALUES (...)")
        conn.execute(stmt, row.to_dict())
```

- Each row is inserted one by one using parameterized SQL.
- The `text()` method avoids SQL injection and formats placeholders as `:column_name`.

Result:

Clean, row-by-row insertion of structured employment data into MySQL.

2. MongoDB Data Insertion – Employment Dataset

Files Used:

Same as above: persons, companies, employment relationships.

Core Logic:

You use **PyMongo** to connect to MongoDB and store Excel rows as JSON-like documents.

Key Code:

```
def insert_data(collection, df):
    records = df.to_dict(orient='records')
    for record in records:
        if not collection.find_one(record):  # Avoid duplicate
            collection.insert_one(record)
```

- Converts each DataFrame row to a dictionary.
- Inserts only if the document doesn't already exist.
- Supports semi-structured and nested data automatically (unlike SQL).

Result:

MongoDB collections (`person`, `company`, `works_in`) now contain flexible documents for each row.

3. Neo4j Data Insertion – Social Network Dataset

Files Used:

- `part2_data/users/users_detailed_100.xlsx`
- `part2_data/relation/connections_10_100.xlsx`

Core Logic:

Uses **Neo4j Python Driver** to insert nodes and edges representing users and `FOLLOWS` relationships.

Key Code:

Create USER node:

```
def create_user(tx, user):
    tx.run("""
        MERGE (u:USER {id: $id})
        SET u.name = $name, ...
    """, **user)
```

- `MERGE` ensures the user is only created if not already existing.
- `SET` updates all attributes.

Create FOLLOWS relationship:

```
def create_follows(tx, follower_id, followed_id):
    tx.run("""
        MATCH (a:USER {id: $follower_id}), (b:USER {id: $followed_id})
        MERGE (a)-[:FOLLOWS]→(b)
    """)
```

- Ensures edges are only added once between nodes.

Result:

A property graph in Neo4j with:

- USER nodes (with profile details)
- FOLLOWS edges modeling connections

4. MySQL Data Insertion – Social Network Dataset

Files Used:

- `part2_data/users/users_detailed_100.xlsx`
- `part2_data/relation/connections_10_100.xlsx`

Core Logic:

Like the employment MySQL script, this loads user and connection data into `users` and `follows` tables.

Key Code:

```
df = df.replace({np.nan: None})  # Handle NULLs
stmt = text(f"INSERT INTO {table_name} (...) VALUES (...)")
```

- Ensures that missing Excel values (`NaN`) are properly converted to SQL `NULL`s.

- Inserts into both `users` and `follows` tables using the same `insert_data()` logic.

Result:

MySQL table `users` is populated with user profiles; `follows` contains social graph edges.

Queries - Part 1

Query 1: Get All Employed Persons with Their Company Info

Function:

```
def get_employed_persons_with_companies():
```

SQL Logic:

```
SELECT p.id AS person_id, p.name AS person_name, p.surname, p.birthdate, p.address, p.country,
       c.id AS company_id, c.name AS company_name, c.founding_date, c.sector
FROM person p
JOIN works_in w ON p.id = w.person_id
JOIN company c ON w.company_id = c.id;
```

Explanation:

- This query uses **INNER JOINs** to return only those persons who are employed.
- It first joins the `person` table to the `works_in` table using the person's ID.
- Then, it joins `works_in` to `company` using the company ID.
- The result includes detailed information from both the `person` and `company` tables, but **only for those who have a valid employment relationship**.

Output:

- Each row represents one employed person and their associated company.
- Unemployed persons are **excluded**.

Query 2: Get All Persons (Including Unemployed) with Company Info (If Any)

Function:

```
def get_all_persons_with_companies():
```

SQL Logic:

```
SELECT p.id AS person_id, p.name AS person_name, p.surname, p.birthdate, p.address, p.country,
       c.id AS company_id, c.name AS company_name, c.founding_date, c.sector
FROM person p
LEFT JOIN works_in w ON p.id = w.person_id
LEFT JOIN company c ON w.company_id = c.id;
```

Explanation:

- This query uses **LEFT JOINs** to ensure **all persons** are included, even if they are not employed.
- If a person is not in the `works_in` table (i.e., unemployed), then the company columns will be `NULL`.
- It is useful for viewing everyone in the system and identifying who is currently unemployed.

Output:

- One row per person.
- If a person is employed, the row includes their company info.
- If a person is unemployed, the company columns are empty (`NULL`).

Query 3: Get All Companies (Including Those Without Employees) with Person Info

Function:

```
def get_all_companies_with_persons():
```

SQL Logic:

```
SELECT c.id AS company_id, c.name AS company_name, c.founding_date, c.sector,
       p.id AS person_id, p.name AS person_name, p.surname, p.birthdate, p.address, p.country
FROM company c
LEFT JOIN works_in w ON c.id = w.company_id
LEFT JOIN person p ON w.person_id = p.id;
```

Explanation:

- This query performs a **LEFT JOIN** starting from the `company` table.
- It returns **all companies**, even those that currently do not employ anyone.
- It attempts to find employees via the `works_in` table and joins their details from the `person` table.
- If a company has no employees, the person fields are `NULL`.

Output:

- One row per company-person pair.
- Companies without employees appear with empty (`NULL`) person fields.

Query 4: Get All Persons and Companies (Matching Both Directions)

Function:

```
def get_all_persons_and_companies():
```

SQL Logic:

```
-- First part: LEFT JOIN from person to company
SELECT
    p.id AS person_id, p.name AS person_name, p.surname, p.birthdate, p.address, p.country,
    c.id AS company_id, c.name AS company_name, c.founding_date, c.sector
FROM person p
LEFT JOIN works_in w ON p.id = w.person_id
LEFT JOIN company c ON w.company_id = c.id

UNION

-- Second part: LEFT JOIN from company to person
SELECT
    p.id AS person_id, p.name AS person_name, p.surname, p.birthdate, p.address, p.country,
    c.id AS company_id, c.name AS company_name, c.founding_date, c.sector
FROM company c
LEFT JOIN works_in w ON c.id = w.company_id
LEFT JOIN person p ON w.person_id = p.id;
```

Explanation:

- This query **combines two separate LEFT JOINs using UNION** to simulate a **FULL OUTER JOIN**.
- **MySQL does not support FULL OUTER JOIN**, so this workaround ensures:
  - All persons are included (even if they don't work anywhere).
  - All companies are included (even if they employ no one).
- It covers every possible pairing between persons and companies, whether or not a relationship exists.

Output:

- All combinations of persons and companies, including:
  - Employed persons and their companies.
  - Unemployed persons.
  - Companies with no employees.

MongoDB Data Setup in Spark

Before running queries:

1. Data is read from MongoDB collections:
   - `person`, `company`, `works_in`
2. `_id` fields are removed to clean the documents.
3. Cleaned data is converted into Spark DataFrames and registered as SQL **temporary views**:

```
person_df.createOrReplaceTempView("person")
company_df.createOrReplaceTempView("company")
works_in_df.createOrReplaceTempView("works_in")
```

This setup allows us to run SQL queries directly on the data as if it were stored in relational tables.

- **Executing SQL queries** on top of these views using `spark.sql()`:

```
df_query5 = spark.sql(query5)
```

The purpose and logic of Queries 5–8 are the
same as Queries 1–4 — but instead of querying MySQL directly, we're using **SparkSQL on MongoDB-derived DataFrames**. So Skipping providing description of same queries again.

MongoDB to Spark RDD Setup

- Data is extracted from MongoDB using PyMongo.
- The `_id` field is removed from each document.
- Data is parallelized into Spark RDDs:
  - `person_rdd`
  - `company_rdd`
  - `works_in_rdd`

Key-value RDDs are also created for efficient lookups:

```
person_kv = person_rdd.map(lambda p: (p['id'], p))
company_kv = company_rdd.map(lambda c: (c['id'], c))
```

Query 9: Employed Persons Together with the Companies They Work In

Objective:

Return only those persons who are employed, along with their company info.

Key Logic:

```
works_by_person = works_in_rdd.map(lambda w: (w['person_id'], w['company_id']))
person_company_grouped = works_by_person.groupByKey()
```

- Group the company IDs each person is connected to.

Then, flatten into individual `(person_id, company_id)` relationships:

```
person_with_companies = person_company_grouped.flatMap(
    lambda x: [(x[0], cid) for cid in x[1]]
)
```

We then use dictionary lookups from pre-collected mappings:

```
person_dict = person_kv.collectAsMap()
company_dict = company_kv.collectAsMap()
```

Finally, enrich each pair:

```
query9 = person_with_companies.map(lambda x: (
    x[0],
    person_dict[x[0]]['name'],
    person_dict[x[0]]['surname'],
```

```
    x[1],
    company_dict[x[1]]['name']
))
```

Result:

A list of `(person_id, person_name, person_surname, company_id, company_name)` for **only employed people**.

Query 10: All Persons with Company Info (If Any)

Objective:

Include all persons in the system. If employed, show their companies; if not, show `None`.

Key Logic:

```
works_by_person = works_in_rdd.map(lambda w: (w['person_id'], w['company_id']))
person_company_list = works_by_person.groupByKey().mapValues(lambda ids: list(ids))
person_to_companies = person_company_list.collectAsMap()
```

Then iterate over all persons:

```
for pid in person_dict.keys():
    person = person_dict[pid]
    company_ids = person_to_companies.get(pid, [None])
    for cid in company_ids:
        company_name = company_dict[cid]['name'] if cid and cid in company_dict else None
```

This guarantees every person is included. Unemployed persons appear with `None` for the company fields.

Query 11: All Companies with Person Info (If Any)

Objective:

Include all companies. If they employ someone, list them; if not, show `None` for the person.

Key Logic:

```
company_person_grouped = works_by_company.groupByKey().mapValues(lambda ids: list(ids))
company_to_persons = company_person_grouped.collectAsMap()
```

Loop through all company IDs:

```
for cid in company_dict.keys():
    company = company_dict[cid]
    person_ids = company_to_persons.get(cid, [None])
    for pid in person_ids:
        person_name = person_dict[pid]['name'] if pid and pid in person_dict else None
```

This simulates a **LEFT JOIN** from company → person.

Result:

A list of:

```
(company_id, company_name, person_id, person_name)
```

Even if a company has no employees, it appears with `None`.

Query 12: All Persons and All Companies (Full Join)

Objective:

Simulate a **FULL OUTER JOIN** between persons and companies, even though RDDs do not support this natively.

Step-by-step Code:

Step 1: Prepare dictionaries of relationships

```
works_by_person_dict = works_in_rdd.map(lambda w: (w['person_id'], w['company_id'])).groupByKey().mapValues(list).collectAsMap()
```

```
works_by_company_dict = works_in_rdd.map(lambda w: (w['company_id'], w['person_id'])).groupByKey().mapValues(list).collectAsMap()
```

Step 2: LEFT JOIN simulation (persons → companies)

```
for pid in person_dict:
    person = person_dict[pid]
    if pid in works_by_person_dict:
        for cid in works_by_person_dict[pid]:
            cname = company_dict[cid]['name'] if cid in company_dict else None
            person_side_data.append((pid, person['name'], person['surname'], cid, cname))
    else:
        person_side_data.append((pid, person['name'], person['surname'], None, None))
```

Step 3: RIGHT JOIN simulation (companies → persons)

```
for cid in company_dict:
    company = company_dict[cid]
    if cid in works_by_company_dict:
        for pid in works_by_company_dict[cid]:
            pname = person_dict[pid]['name'] if pid in person_dict else None
            psurname = person_dict[pid]['surname'] if pid in person_dict else None
            company_side_data.append((pid, pname, psurname, cid, company['name']))
    else:
        company_side_data.append((None, None, None, cid, company['name']))
```

Step 4: Combine and deduplicate

```
combined_data = list(set(person_side_data + company_side_data))
query12_data = sc.parallelize(combined_data)
```

Result:

This output contains:

- Employed persons and their companies
- Unemployed persons (with `None` as company)
- Companies with no employees (with `None` as person)

PART2

Query 1: Neo4j – Find Reachable Users via Cypher Query

Objective:

Return all users that a given user can reach through one or more `FOLLOWS` relationships. In other words, users whose content the starting user can view (directly or through friends-of-friends).

Key Code:

```
MATCH (start:USER {id: $user_id})-[:FOLLOWS*1..]→(reachable:USER)
RETURN DISTINCT reachable.id AS visible_user_id
```

Explanation:

- `[:FOLLOWS*1..]` means **one or more** `FOLLOWS` **hops**.
- It performs a **recursive traversal** starting from the given `user_id`.
- `DISTINCT` avoids duplicate user entries.
- Results are collected and returned as a Python list using Neo4j's Python driver.

Query 2: Spark + GraphFrames – BFS on Social Graph

Objective:

Perform the same operation as Query 1 (reachable users) but using **GraphFrames** in **Spark**, with data stored in **MySQL**.

Key Steps:

1. Load data and prepare vertices/edges:

```
users_df = users_full.select("id")
follows_df = spark.read.jdbc(...).selectExpr("follower_id as src", "followed_id as dst")
```

2. Build GraphFrame:

```
g = GraphFrame(users_df, follows_df)
```

3. Run Breadth-First Search:

```
paths = g.bfs(
    fromExpr = f"id = {start_user_id}",
    toExpr   = f"id != {start_user_id}",
    maxPathLength = 10
)
```

- This will traverse from the start user to all reachable users (up to 10 hops).

4. Extract IDs of reachable users:

```
reachable_ids = paths.select("to.id").distinct()
```

5. Join with full user info:

```
reachable_users = reachable_ids.join(users_full, reachable_ids.reachable_id == users_full.id)
```

Query 3: Spark + RDD – Custom BFS Traversal

Objective:

Simulate the same graph traversal manually using **RDDs and set operations**.

Key Code:

1. Load users and edges as RDDs:

```
users_rdd = users_df.rdd.map(lambda row: (row.id, row.name))
edges_rdd = follows_df.rdd.map(lambda row: (row.src, row.dst))
```

2. Custom BFS function:

```
def get_reachable(start_id, edges):
    visited = set()
    frontier = {start_id}

    while frontier:
        new = edges.filter(lambda e: e[0] in frontier).map(lambda e: e[1]).distinct().collect()
        new_set = set(new) - visited - frontier
        if not new_set:
            break
        visited.update(frontier)
        frontier = new_set
    return visited
```

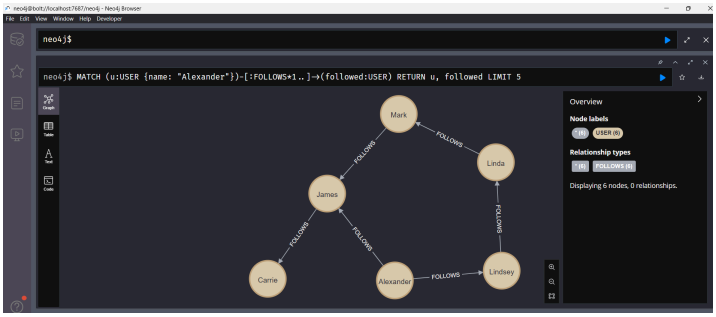
- `frontier` holds the set of current nodes to explore.
- For each round, find all direct connections from the frontier.
- `visited` ensures no cycles or duplicate work.

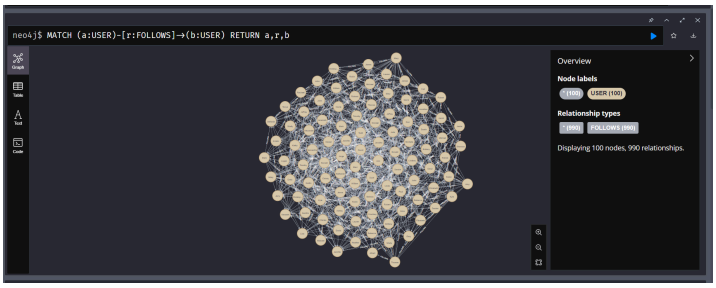
3. Lookup names from ID:

```
user_map = dict(users_rdd.collect())
reachable = [(uid, user_map.get(uid, "<unknown>")) for uid in sorted(reachable_ids)]
```

Output:

- Prints reachable user IDs and their names.
- Matches functionality of Query 1 and Query 2, implemented at a lower level.





Performance Comparison Report

This analysis compares the **execution times** of various queries implemented using different technologies (MySQL, Spark SQL, Spark RDD, Neo4j, GraphFrames) across increasing dataset sizes and relationship densities.

Part 1: Person–Company Queries

We compared 12 queries across three scenarios:

Persons	Companies	Occupation Rate
100	10	25%
1,000	10	25%
10,000	100	50%

Observations:

- **MySQL (Query 1–4)** scales extremely well with person count.
  - Query times remain under 0.5 seconds, even with 10,000 persons.
  - Ideal for small to mid-size datasets with tabular structure.
- **Spark SQL (Query 5–8)** maintains relatively stable times (16–34 seconds).
  - Good scalability but incurs significant overhead due to job startup, even on small data.
  - Graph-heavy queries (Query 8) take the longest.
- **Spark RDDs (Query 9–12)** are the slowest but flexible.
  - More memory consumption, custom logic.
  - Execution time grows with dataset size.

Performance Bottlenecks:

- Spark (especially RDD) becomes less efficient as relationships grow.
- MySQL performs best in terms of raw speed and simplicity, but lacks distributed capabilities.

The graph shows the execution times of person–company queries across increasing dataset sizes (100, 1,000, and 10,000 persons). MySQL-based queries (Query1–4) perform exceptionally well, with times remaining under 0.5 seconds even for the largest dataset, making them ideal for structured and smaller-scale data. Spark SQL queries (Query5–8) are consistent in performance, but their execution times remain around 16 to 32 seconds due to Spark's inherent overhead, regardless of dataset size. RDD-based queries (Query9–12) are the slowest, ranging from 23 to 35 seconds, but offer flexibility for custom logic. Overall, MySQL is the most efficient for fast execution, while Spark provides scalability at the cost of speed.

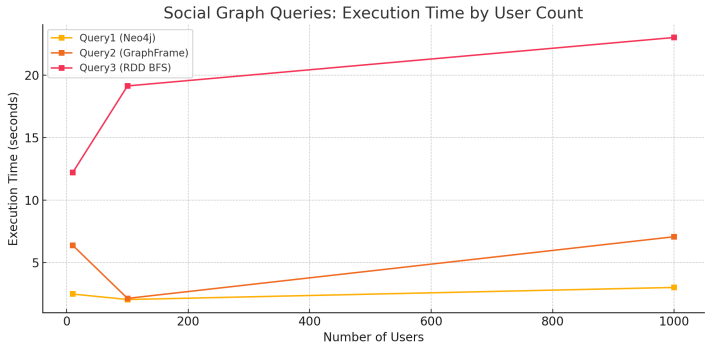
Part 2: User–User Social Graph Queries

We measured Neo4j, GraphFrames, and Spark RDD on follow relationship datasets of:

Users	% Connections	Total Edges
10	25%	22
100	10%	990
1,000	~1%	9,999

Observations:

- **Neo4j (Query 1)** is fast and predictable, even for larger user graphs.
  - Best for recursive relationships.
  - Efficient up to 1000 nodes in-memory.
- **GraphFrames (Query 2)** shows inconsistent performance:
  - Fast for 100 users, but slow for small or large sets due to BFS overhead.
  - Best suited for mid-size networks with optimized joins.
- **Spark RDD BFS (Query 3)** has the highest latency.
  - Full control, but high cost of `.collect()` and multiple iterations.
  - Performance degrades quickly as graph size grows.



The graph shows how execution time increases for social graph queries as the number of users grows.
Neo4j (yellow) performs the best, staying fast even at 1000 users. **GraphFrames (orange)** is efficient for medium datasets but less consistent. **RDD BFS (red)** is the slowest, with time rising sharply as user count increases. This confirms Neo4j is best for graph traversal, while RDDs are flexible but slow.

Conclusion:

1. **For Relational Joins** (person ↔ company):
   - Use **MySQL** for up to 10,000 records: fast, simple, and efficient.
   - Use **Spark SQL** if data needs distributed processing or integration with pipelines.
2. **For Graph Traversals**:
   - Use **Neo4j** when working with complex user relationships and deep follow paths.
   - Use **GraphFrames** when embedded into Spark pipelines, but avoid small datasets.
   - Avoid RDDs unless algorithm customization is critical.
3. **Scalability**:
   - MySQL scales well with row-based data.
   - Spark introduces overhead but supports large-scale processing.
   - Neo4j is ideal for highly connected datasets.